ACESS

Adaptive Cognitive & Educational Skill Support Platform

Technical Documentation

System Architecture · Database Schema · Feature Reference · Deployment Guide

Version	1.0.0 (May 2026)
Framework	Next.js 16 · Supabase · TypeScript 5
Live URL	https://acess-tau.vercel.app
Repository	https://github.com/aliff1024/ACESS

Prepared by the ACESS Engineering Team

Table of Contents

1 · Platform Overview

ACESS (Adaptive Cognitive & Educational Skill Support) is an accessibility-first Learning Management System (LMS) designed to serve learners with cognitive and learning differences. The platform adapts its visual presentation, content delivery, and navigation to meet individual accessibility needs while providing educators and administrators with robust tools for content creation, student management, and analytics.

User Roles

Role	Capabilities
Learner	Browse and enroll in courses · Take quizzes · Track progress · Earn certificates · Bookmark favourites · Adjust accessibility settings
Educator	Create and publish courses, lessons, and quizzes · Monitor student progress · Access per-course analytics · Upload media assets
Admin	Manage system-wide courses and chapters · Issue/revoke certificates · View platform analytics · Manage all user accounts · Control content moderation

Technology Stack

Layer	Technology	Notes
Framework	Next.js 16.2.3 (App Router)	Full-stack React framework with SSR, file-based routing
Language	TypeScript 5	Strict typing across all modules
Styling	Tailwind CSS v4 + shadcn/ui	Utility-first CSS; 45+ radix-nova UI components
Database	Supabase (PostgreSQL)	Managed Postgres with built-in RLS and realtime
Auth	Supabase Auth	Email/password with cookie-based session management
Email	Nodemailer + Gmail SMTP	Custom transactional email for password resets
Charts	Recharts	Analytics dashboards for educators and admins
Animation	Framer Motion	Page transitions and micro-interactions
Forms	React Hook Form + Zod	Validated forms with schema-based error handling
Icons	Lucide React	Consistent icon set across the application
Carousel	Embla Carousel	Lesson asset and content carousels

Toasts	Sonner	Non-intrusive notification toasts
Hosting	Vercel	Automatic deploys from GitHub; edge CDN

2 · System Architecture

ACESS follows a client-centric architecture built on the Next.js App Router. The vast majority of database interactions occur through client-side Supabase calls managed by three purpose-built API modules. Server Components and API Routes are used selectively—primarily for auth middleware and the password-reset email flow. This design keeps the codebase lean while leveraging Supabase's Row-Level Security (RLS) for data isolation.

Architectural Principles

No React Query / SWR	All data fetching uses useState + useEffect with direct Supabase calls, avoiding additional caching layers.
No Server Actions	All mutations execute through client-side Supabase SDK calls, keeping the mutation pattern uniform.
RLS-first security	Supabase RLS policies enforce data isolation at the database level; the application layer adds role routing.
Service Role for admin ops	Password resets and cross-user recommendation generation bypass RLS using a service role key, never exposed to the browser.
CSS data attributes	Accessibility settings (theme, font, spacing) are written as data-* attributes on the element, allowing CSS to respond without JS re-renders.
Custom SMTP	Bypasses Supabase's built-in email for full control over email design, sender identity, and delivery reliability.

API Modules

All Supabase queries are centralised in three TypeScript modules. Each function calls ensureUserId() to resolve the authenticated user before executing any query.

Module	Size (est.)	Responsibility
src/lib/learner-api.ts	~1,270 lines	Learner profile, course browsing, lesson progress, quiz attempts, certificate retrieval, recommendations, favourites, accessibility settings
src/lib/educator-api.ts	~838 lines	Course/lesson/quiz CRUD, student roster, per-course analytics, media asset uploads to Supabase Storage

src/lib/admin-api.ts	~851 lines	System course management, chapter/milestone control, lesson templates, user management, platform-wide analytics, certificate issuance
src/lib/recommendation-engine.ts	Utility	Uses service role key to bypass RLS for cross-user course recommendation generation

Supabase Client Instances

File	Client Type	Key Used	Used For
src/lib/supabase.ts	createBrowserClient()	Anon key	All browser-side queries; respects RLS
src/lib/supabase-server.ts	createServerClient() via @supabase/ssr	Anon key	Server Components and API Route handlers
src/lib/recommendation-engine.ts	createClient()	Service role key	Cross-user operations that must bypass RLS

3 · Data Flow & Request Lifecycle

Page Request Flow

Every page request passes through Next.js middleware before reaching the application layer. The middleware validates the Supabase session, checks the user role, and either allows the request or issues a redirect. Only after passing the middleware gate does the browser receive HTML.

Step	Layer	Action
1	Browser	User navigates to a protected URL (e.g. /learner/dashboard)
2	Middleware	proxy.ts intercepts the request; createServerClient() reads the session cookie
3	Supabase Auth	getUser() validates the JWT; returns user object or null
4a	Middleware	If no user → redirect to /login
4b	Middleware	If wrong role → redirect to /access-denied
4c	Middleware	If valid → NextResponse.next() with refreshed session cookie
5	Next.js Server	Server Component renders initial HTML with any server-fetched data
6	Browser	React hydrates; client components mount and begin their own Supabase fetches
7	Supabase DB	Client-side queries execute via createBrowserClient(); RLS filters rows by user_id
8	Browser	UI updates with fetched data; loading states resolve

Client-Side Mutation Flow

User actions (form submissions, button clicks) follow a consistent pattern across all features:

- **User Action:** Button click or form submit triggers an async event handler
- **Optimistic UI:** Loading spinner activates; submit button disables to prevent double submission
- **API Call:** The relevant learner-api / educator-api / admin-api function is called
- **Supabase Query:** The SDK executes the INSERT/UPDATE/DELETE with RLS enforcement
- **Response Handling:** On success: toast.success() fires and local state updates; on error: toast.error() with the error message
- **State Sync:** If needed, affected list/data is re-fetched to reflect the server state

4 · Provider & Context Architecture

Global state is managed by three nested React Contexts defined in src/app/providers.tsx. The nesting order is deliberate: Auth must be outermost because the inner providers depend on user identity to load personalised settings. SessionTimeout is innermost as it only monitors activity.

```
// src/app/providers.tsx — nesting order
// Supabase session, user metadata, preview mode
// Font size, theme, TTS, keyboard nav → CSS data-* on
// en/ms locale, t() translation function, persisted to DB
// 15-min inactivity → 30s warning modal → auto-logout
{children}
```

Provider	Hook	Exposed State / Methods
AuthProvider	useAuth()	user, isLoading, isAuthenticated, signOut(), preview, enterPreview(), exitPreview()
AccessibilityProvider	useAccessibility()	All accessibility settings object + updateSetting() + applySettings()
LanguageProvider	useLanguage()	t(key) translation function, locale (en/ms), setLocale()
SessionTimeout	—	No hook; renders warning modal + auto-logout countdown internally

Preview Mode

AuthProvider includes a preview mode for the public landing page. Visitors can click a role selector to see what each dashboard looks like without creating an account. enterPreview(role) injects a synthetic user object, and exitPreview() clears it. Preview mode is signalled via the preview boolean so components can disable destructive actions (mutations are no-ops in preview).

5 · Route Architecture & Middleware

Public Routes

The following paths bypass authentication middleware and are accessible without a session:

- /
- /login
- /signup
- /access-denied
- /forgot-password
- /reset-password
- /api/auth/forgot-password
- /api/auth/reset-password

Protected Routes

URL Prefix	Required Role	Admin Override	Notes
/learner/*	learner	Yes	First-time learners redirected to /learner/onboarding
/educator/*	educator	Yes	Course creation, student views, analytics
/admin/*	admin	N/A	Full platform control; cannot be accessed by other roles
/profile/*	Any authenticated	Yes	User profile and settings; all roles
/api/*	Any authenticated	Yes	All API routes require a valid session

Middleware Logic

The middleware (src/proxy.ts) is a Next.js edge function that intercepts every request matching the configured pattern. It uses createServerClient() from @supabase/ssr to read the session cookie server-side, validating the JWT without a round-trip to Supabase Auth. Role resolution uses user.user_metadata.role, which is set at signup and stored in auth.users.

> Admin users have a bypass flag — they can access all role-specific routes. If a user's role does not match the required prefix, they are redirected to /access-denied rather than /login to distinguish "wrong role" from "not logged in".

Adding New Routes

When adding new public routes (pages that should not require login), append the path to the PUBLIC_ROUTES array in src/proxy.ts. New protected routes are automatically guarded if they fall under an existing prefix (/learner, /educator, /admin). For new top-level protected sections, add a corresponding entry to ROLE_PREFIXES.

6 · Database Schema

All tables reside in the public schema of a managed Supabase PostgreSQL instance. Row-Level Security (RLS) is enabled on all tables. The database is organised into five logical groups: core user and course tables, content tables (lessons, assets, templates), quiz tables, feature tables (recommendations, certificates, notifications), and accessibility/helper tables.

Core Tables

users

Column	Type	Notes
id	uuid PK	Matches auth.users.id
email	varchar UNIQUE	
full_name	text	
role	varchar	learner / educator / admin
is_active	boolean	
email_verified_at	timestamptz	
last_login_at	timestamptz	
created_at	timestamptz	
deleted_at	timestamptz	Soft-delete flag; queries filter .is(deleted_at, null)

courses

Column	Type	Notes
id	uuid PK	
created_by	uuid FK→users	
title	varchar	
slug	varchar UNIQUE	URL-safe identifier
description	text	
difficulty_level	varchar	beginner / intermediate / advanced

category	text	
thumbnail_url	varchar	
status	varchar	draft / pending_review / published / archived
course_type	text	educator / system
system_course	boolean	Managed by admin
guided_learning_enabled	boolean	Enables assisted learning mode
recommended_age_group	text	
published_at	timestamptz	

enrollments

Column	Type	Notes
id	uuid PK	
user_id	uuid FK→users CASCADE	
course_id	uuid FK→courses CASCADE	
status	varchar	active / completed / dropped
enrolled_at	timestamptz	
completed_at	timestamptz	Set when all lessons completed and final quiz passed
UNIQUE	(user_id, course_id)	Prevents duplicate enrollments at the DB level

Content Tables

lessons

Column	Type	Notes
id	uuid PK	
course_id	uuid FK	
chapter_id	uuid FK	Optional grouping within course
title	varchar	

content_html	text	Main lesson body; rendered as sanitised HTML
video_url	varchar	Optional embedded video
transcript	text	For TTS and captions
sequence_order	integer	UNIQUE per course; defines lesson order
status	varchar	draft / published
prerequisite_lesson_id	uuid FK	Optional dependency gate
estimated_duration	integer	Minutes; shown to learner before starting
lesson_type	text	standard / video / quiz / practice / reading / assessment
simplified_summary	text	Shown in assisted learning mode
accessibility_notes	text	Educator hints for accessible delivery
learning_objectives	text	Displayed at lesson start

lesson_progress

Column	Type	Notes
id	uuid PK	
enrollment_id	uuid FK CASCADE	
lesson_id	uuid FK CASCADE	
is_viewed	boolean	
view_count	integer	Incremented on each lesson visit
first_viewed_at	timestamptz	
last_viewed_at	timestamptz	
time_spent_learning	integer	Seconds; accumulated across sessions
assisted_learning_mode	boolean	Whether simplified mode was active
UNIQUE	(enrollment_id, lesson_id)	

Quiz Tables

Quizzes have a 1:1 relationship with lessons (a quiz lesson has exactly one quiz record). Questions support multiple_choice and scenario types. Attempts are tracked per enrollment, enabling max-attempt enforcement and attempt history.

Table	Key Columns	Notes
quizzes	lesson_id (UNIQUE FK), time_limit_seconds, max_attempts, pass_threshold_pct	One quiz per quiz-type lesson; 0 = unlimited for time/attempts
quiz_questions	quiz_id FK, question_text, question_type, sequence_order	multiple_choice or scenario; ordered by sequence_order
quiz_options	question_id FK, option_text, is_correct, sequence_order	Multiple options per question; only one is_correct per question
quiz_attempts	enrollment_id FK, quiz_id FK, attempt_number, score_pct, result	result: pass / fail / in_progress; attempt_number auto-increments in app code
quiz_answers	attempt_id FK, question_id FK, selected_option_id FK, is_correct	UNIQUE(attempt_id, question_id) prevents duplicate answers per attempt

Feature Tables

Table	Purpose	Key Columns
recommendations	AI-driven next-lesson suggestions per enrollment	enrollment_id, recommended_lesson_id, difficulty_tier (revision/standard/advanced), trigger_reason, is_acknowledged
certificates	Completion certificates; one per enrollment	enrollment_id (UNIQUE), reference_code (UNIQUE), pdf_url, status (issued/revoked)
notifications	In-app notifications triggered by DB events	user_id, type, title, body, metadata (jsonb), is_read
course_favorites	Learner course bookmarks	user_id + course_id (UNIQUE pair)
course_chapters	Optional lesson grouping within a course	course_id, title, sequence_order (UNIQUE per course)
course_milestones	Progress milestones with completion percentage thresholds	course_id, required_completion_pct (0–100), sequence_order

Accessibility & Profile Tables

Table	Purpose

user_accessibility_settings	Per-user disability type, font size, theme, line spacing, TTS, captions, reduced motion, dyslexia font, preferred language
user_profiles	Extended profile: username, avatar_url, phone_number, birth_date, bio, country, preferred_language
user_notification_settings	Email and push notification preferences per user
child_accounts	Links a guardian (educator/parent) to a child learner for progress monitoring
password_reset_tokens	Temporary one-time tokens for the custom password reset flow; expires_at + used flag
lesson_templates	Reusable lesson structures created by educators; can be public or private

Database Triggers (Notification Automation)

Five PostgreSQL triggers automatically generate notification records when key events occur, eliminating the need for application-layer notification logic:

Trigger	Event	Fires When
notify_on_enrollment	INSERT on enrollments	Learner enrols → notify course creator
notify_on_lesson_progress	INSERT on lesson_progress	First lesson view → notify educator
notify_on_quiz_attempt	INSERT on quiz_attempts	Quiz attempted → notify educator
notify_on_lesson_added	INSERT on lessons	New lesson published → notify enrolled learners
notify_on_course_published	UPDATE on courses	Course status set to published → notify enrolled learners

RLS Policy Summary

Actor	Permitted Operations
Authenticated learner	SELECT own rows from users, enrollments, lesson_progress, quiz_attempts, recommendations, certificates, favorites, accessibility settings
Educator	CRUD own courses, lessons, quizzes; SELECT enrolled students' progress on their courses
Admin	SELECT and UPDATE all users; full CRUD on system courses, chapters, milestones, certificates
Service role	Bypasses all RLS; used for password resets, recommendation generation, and admin password updates

7 · Authentication & Authorization

7.1 Login

Login is handled by supabase.auth.signInWithPassword(). On success, Supabase returns a session containing access_token and refresh_token, which are stored in cookies by the SSR client. The application reads user.user_metadata.role to determine the post-login redirect target.

Role	Redirect Target	Special Case
learner	/learner	If no learner_profiles row exists → redirect to /learner/onboarding
educator	/educator	—
admin	/admin	—

7.2 Signup

New accounts are created with supabase.auth.signUp(). The full_name and role fields are stored in user_metadata. A database trigger on auth.users automatically creates a corresponding row in public.users. Validation uses React Hook Form with Zod (minimum 6-character password, valid email format). Role selection is limited to learner or educator; admin accounts are created directly by an existing admin.

7.3 Session Management

The AuthProvider subscribes to onAuthStateChange() and updates React context whenever the Supabase session changes. The SSR client automatically refreshes expired access tokens using the refresh_token cookie on each server request, keeping long-lived sessions seamless.

7.4 Session Timeout

A SessionTimeout component wraps all children and monitors four browser events: mousemove, keydown, touchstart, and scroll. After 15 minutes of inactivity, a modal appears with a 30-second countdown. Clicking "Stay logged in" resets the timer. If the countdown reaches zero, supabase.auth.signOut() is called automatically.

7.5 Password Reset Flow

ACESS implements a custom password reset flow independent of Supabase's built-in email system, providing full control over email content and delivery reliability.

- **Step 1 — Request:** User submits /forgot-password form with their email address.
- **Step 2 — Token generation:** POST /api/auth/forgot-password generates a 64-char hex token via crypto.randomBytes(32), stores it in password_reset_tokens with a 1-hour expiry, and sends a reset email via Gmail SMTP (Nodemailer).
- **Step 3 — Email delivery:** The email contains a one-time link: /reset-password?token=XXX&email;=YYY. The reset link is valid for 60 minutes.
- **Step 4 — Password update:** POST /api/auth/reset-password validates the token (not used, not expired), calls adminSupabase.auth.admin.updateUserById() with the new password (bypasses RLS), and marks the token as used.
- **Step 5 — Redirect:** User is redirected to /login with a success toast.

> The API always returns { sent: true } regardless of whether the email exists, preventing user enumeration. Tokens are single-use and expire after one hour. The service role key required for the admin password update is server-side only and never exposed to the browser.

8 · Accessibility System

Accessibility is a first-class concern in ACESS, not an afterthought. The platform is specifically designed for learners with dyslexia, ADHD, and mild cognitive impairments. Settings persist to the database and apply on every device the user logs in from.

8.1 Settings Persistence

On mount, AccessibilityProvider fetches the user's row from user_accessibility_settings. Any change calls updateSetting(), which debounces the Supabase UPDATE to avoid excessive writes. Settings are also written as data-* attributes on the element so that CSS can respond instantly without React re-renders.

8.2 Available Settings

Setting	Options / Type	Effect
preferred_font_size	small / medium / large / xlarge	Scales base font size via CSS variable
preferred_theme	light / dark / high_contrast / soft	Switches Tailwind theme; high_contrast meets WCAG AA
line_spacing	normal / relaxed / loose	Adjusts line-height CSS variable
preferred_font	default / serif / sans_serif / dyslexia	dyslexia option loads OpenDyslexic font
dyslexia_friendly_font	boolean	Shortcut to enable OpenDyslexic
reduced_motion	boolean	Disables Framer Motion animations; respects prefers-reduced-motion
simplified_ui	boolean	Hides decorative elements; increases hit targets
tts_enabled	boolean	Enables Web Speech API text-to-speech on lesson content
captions_enabled	boolean	Displays lesson video captions/transcripts
screen_reader_optimized	boolean	Adds additional ARIA labels and removes visual-only decorations
keyboard_navigation_enabled	boolean	Enhances visible focus rings and keyboard shortcuts

disability_type	dyslexia / adhd / mild_cognitive_impairment / other	Stored for analytics; may auto-apply presets
preferred_language	en / ms	Syncs with LanguageProvider locale

8.3 Text-to-Speech

When tts_enabled is true, lesson content paragraphs receive click/focus handlers that call the Web Speech API (window.speechSynthesis.speak()). The system uses the browser's built-in TTS engine, requiring no third-party service. Users can pause, resume, and stop playback.

8.4 Assisted Learning Mode

Enabled per-enrollment when guided_learning_enabled is set on the course. In this mode, each lesson shows its simplified_summary field instead of the full content_html. Progress is still tracked normally. The lesson_progress.assisted_learning_mode flag records which mode was active when the lesson was viewed.

9 · Internationalisation (i18n)

ACESS supports two languages: English (en) and Bahasa Malaysia (ms). Translations are stored as TypeScript objects in src/locales/en.ts and src/locales/ms.ts, enabling type-safe key lookups. The active locale persists to user_accessibility_settings.preferred_language in the database.

t() Translation Function

LanguageProvider exposes the t(key) function which resolves a dot-notation key (e.g. "dashboard.welcome") against the current locale's translation object. If a key is missing in the active locale, it falls back to the English equivalent.

Locale Switching

Users can toggle their language in profile settings. setLocale(locale) updates the LanguageProvider state, persists the value to the database, and re-renders all translated strings. No page reload is required — the translation object swap is synchronous.

10 · Course & Content Management

10.1 Course Lifecycle

Status	Who Sets It	Visibility
draft	Educator (on creation)	Only visible to the creating educator
pending_review	Educator (submits for review)	Visible to admins for review
published	Admin (approves) or Educator (for auto-publish)	Visible to all learners; triggers notification to enrolled learners
archived	Educator or Admin	Hidden from course catalogue; enrolled learners retain access

10.2 Content Hierarchy

A course can optionally be organised into chapters (course_chapters), which group lessons for navigation. Lessons within a chapter are ordered by sequence_order. Chapters themselves have a sequence_order within the course. Milestones (course_milestones) mark significant completion checkpoints (e.g. 25%, 50%, 75%) and can trigger badge awards or unlock content.

```
Course
■■■ Chapter 1 (sequence_order: 1)
■ ■■■ Lesson A (sequence_order: 1)
■ ■■■ Lesson B (sequence_order: 2) ← prerequisite_lesson_id: Lesson A
■■■ Chapter 2 (sequence_order: 2)
■ ■■■ Lesson C (sequence_order: 3)
■■■ Milestone: "Halfway!" (required_completion_pct: 50)
```

10.3 Lesson Types

Type	Description
standard	Text-based lesson with rich HTML content; supports embedded images and code blocks
video	Video-primary lesson with optional transcript and accessibility notes
quiz	Assessment lesson with an associated quiz record; must be passed to count toward completion
practice	Interactive exercise lesson; completion tracked by checkpoint flags

reading	Long-form reading material with reading time estimate
assessment	Formal graded assessment; may count toward certificate eligibility

10.4 Media & Asset Management

Educators upload media via the Supabase Storage bucket course-assets. The bucket policy allows upload-only for authenticated educators (no public read or delete). Uploaded file URLs are stored in lesson_assets (for attachments) or directly in lessons.video_url / courses.thumbnail_url. A Fallback Image component handles broken image URLs gracefully.

10.5 Lesson Templates

Educators can save any lesson structure as a reusable template (lesson_templates). Templates capture the lesson_type, content_html, and estimated_duration. Public templates (is_public: true) are accessible to all educators on the platform, enabling knowledge sharing and reducing content creation time.

11 · Learning Engine

11.1 Enrollment

Learners enroll via a single button on the course detail page. The enrollment INSERT is guarded by a UNIQUE(user_id, course_id) constraint, making double-enrollment impossible. On enrollment, the notify_on_enrollment trigger fires, creating a notification for the course creator. The initial enrollment status is active.

11.2 Progress Tracking

Lesson progress is written to lesson_progress on first view. Subsequent visits increment view_count and update last_viewed_at. Time spent is accumulated in time_spent_learning (seconds) and periodically flushed to the database during the lesson session. Course completion percentage is calculated as:

> completion_pct = (count of lessons where is_viewed = true) / (total published lessons in course) × 100

When completion_pct reaches 100% and all quiz-type lessons have a passing attempt, the enrollment.status is set to completed and a certificate is automatically issued.

11.3 Quiz Engine

Each quiz-type lesson has exactly one quiz record. Learners start an attempt (quiz_attempts INSERT with result: in_progress), answer each question (quiz_answers INSERTs), then submit. On submission:

- score_pct is calculated as (correct answers / total questions) × 100.
- result is set to pass if score_pct ≥ pass_threshold_pct (default 80), otherwise fail.
- If max_attempts > 0 and the learner has reached the limit, further attempts are blocked.
- If time_limit_seconds > 0, the UI runs a countdown timer; on expiry, the attempt is auto-submitted.
- All attempts are stored, giving educators a full attempt history per student.

11.4 Certificates

Upon course completion, the system generates a certificate record with a unique reference_code (UUID-based, human-readable). A PDF certificate is generated server-side and stored in Supabase Storage; the URL is saved in certificates.pdf_url. Certificates have a status of issued or revoked. Admins can revoke

certificates; revoked certificates are invalidated on the public verification page.

11.5 Lesson Checkpoints

Within a lesson, educators can define checkpoints (lesson_checkpoints) of types: reflection, practice, quiz, activity, or milestone. Learner responses are stored in learner_checkpoints per enrollment. Checkpoints must be completed for the lesson to count as finished in some course configurations.

12 · Recommendation Engine

ACESS includes an adaptive recommendation system that suggests next lessons based on learner performance. Recommendations are stored in the recommendations table and surfaced in the learner dashboard.

Difficulty Tiers

Tier	Trigger Condition	Purpose
revision	Quiz score below pass threshold on first attempt	Suggest easier or prerequisite content for reinforcement
standard	Normal progression through course	Next lesson in sequence; maintains learning momentum
advanced	Quiz score above 90% with minimal time spent	Challenge learner with extension or advanced content

Implementation Notes

- The recommendation engine uses a service role Supabase client (createClient() with SUPABASE_SERVICE_ROLE_KEY) to query cross-user data without RLS restrictions.
- Recommendations are generated server-side when a quiz attempt is submitted or a lesson is completed.
- Learners can acknowledge (dismiss) a recommendation by setting is_acknowledged: true.
- trigger_reason stores a human-readable explanation (e.g. "Quiz score was 60% — review this concept") shown in the recommendation card.

13 · Notification System

Notifications are generated automatically by PostgreSQL triggers and surfaced in the application via real-time Supabase subscriptions. Each notification targets a specific user and carries a type, title, body, and optional metadata JSON blob.

Type	Recipient	Trigger
enrollment	Course creator (educator)	Learner enrols in the course
lesson_completed	Educator	Learner views a lesson for the first time
quiz_completed	Educator	Learner submits a quiz attempt
lesson_added	Enrolled learners	Educator publishes a new lesson to their course
course_published	Enrolled learners	Course status changes to published

Real-Time Delivery

The notification bell component subscribes to the notifications table via supabase.channel().on('postgres_changes', ...) for real-time updates. The unread count badge updates immediately when a new notification row is inserted. Clicking a notification marks it as read (is_read: true). Users can also configure email/push preferences in user_notification_settings to opt out of specific notification types.

14 · Educator & Admin Features

Educator Capabilities

• **Course Builder:** Rich editor for course metadata, thumbnail upload, difficulty level, category, and age group. Courses start in draft status.

• **Lesson Editor:** HTML content editor with lesson type selector, video URL input, transcript field, simplified summary for accessibility mode, and learning objectives.

• **Quiz Builder:** Add multiple-choice or scenario questions with ordered options. Set pass threshold, time limit, and max attempts.

• **Chapter & Milestone Manager:** Drag-and-order chapters and milestones. Set completion percentage triggers for milestones.

• **Student Roster:** View all enrolled learners with progress percentages, quiz scores, and time spent. Individual learner detail view available.

• **Analytics Dashboard:** Per-course charts: enrolment trend, completion rate, quiz pass rate, average score, and average time to complete. Powered by Recharts.

• **Asset Uploader:** Upload PDF, image, or video assets to Supabase Storage; attach to lessons as lesson_assets.

• **Template Library:** Save any lesson as a reusable template. Browse and import public templates from other educators.

Admin Capabilities

• **System Course Management:** Create and manage system courses (course_type: system) that appear for all learners regardless of educator ownership.

• **User Management:** View, activate, deactivate, and soft-delete user accounts. Reset user passwords directly. View any user's full profile.

• **Certificate Control:** Issue certificates manually, revoke issued certificates, and view the complete certificate registry with reference codes.

• **Platform Analytics:** Cross-platform metrics: total users by role, total courses, average completion rate, most popular courses, and platform growth over time.

• **Content Moderation:** Review courses in pending_review status; approve to publish or return with feedback.

• **Chapter & Milestone Control:** Admin can manage chapters and milestones on system courses; educators manage their own.

15 · API Module Reference

All three API modules follow an identical function pattern:

```
async function fetchSomething(): Promise {
const userId = await ensureUserId() // throws if no session
const { data, error } = await supabase
.from('table')
.select('...')
.eq('user_id', userId)
if (error) throw error
return data
}
```

learner-api.ts — Key Functions

Function	Description
getLearnerProfile()	Fetches user profile, accessibility settings, and active enrollments
enrollInCourse(courseId)	Creates enrollment record; handles UNIQUE constraint conflict gracefully
getLessonProgress(enrollmentId)	Returns all lesson_progress rows for an enrollment
markLessonViewed(lessonId, enrollmentId)	UPSERTs lesson_progress; increments view_count
submitQuizAttempt(quizId, answers)	Scores answers, INSERTs attempt and answer rows, returns score
getMyCertificates()	Returns all issued certificates for the current learner
getRecommendations(enrollmentId)	Returns unacknowledged recommendations ordered by created_at
toggleFavorite(courseId)	Upserts or deletes course_favorites row; returns new boolean state
updateAccessibilitySettings(settings)	UPSERTs user_accessibility_settings; applies CSS vars

educator-api.ts — Key Functions

Function	Description
createCourse(data)	INSERTs course with status: draft; returns new course id
updateLesson(lessonId, data)	UPDATEs lesson fields; handles sequence_order reordering

createQuiz(lessonId, data)	INSERTs quiz + questions + options in a single transaction-like sequence
getStudentProgress(courseId)	Returns all enrollments with join to lesson_progress for analytics
uploadAsset(file, lessonId)	Uploads to course-assets bucket; INSERTs lesson_assets row
getCourseAnalytics(courseId)	Aggregates enrolment, completion, and quiz stats for dashboard charts

admin-api.ts — Key Functions

Function	Description
getAllUsers(filters)	Returns paginated user list with role filter, search, and active status
updateUserStatus(userId, active)	Sets is_active; effectively enables/disables login
issueCertificate(enrollmentId)	Creates certificate record with generated reference_code and PDF
revokeCertificate(certId)	Sets certificates.status = revoked
getSystemCourses()	Returns all courses where system_course = true
getPlatformAnalytics()	Cross-role aggregate metrics for the admin dashboard

16 · Frontend Component System

ACESS uses shadcn/ui, built on Radix UI primitives with Tailwind CSS and class-variance-authority (cva). All components are WAI-ARIA compliant and support keyboard navigation out of the box.

Component Categories

Category	Components
Layout	Card, Separator, Sidebar, Sheet, Drawer
Navigation	Breadcrumb, NavigationMenu, Tabs, Pagination, Menubar
Forms	Button, Input, Textarea, Checkbox, RadioGroup, Select, Switch, Slider, Calendar
Overlays	Dialog, AlertDialog, Popover, Tooltip, DropdownMenu, Command
Feedback	Alert, Badge, Progress, Skeleton, Toast (Sonner)
Data Display	Table, Avatar, Accordion, Chart (Recharts wrapper)
Media	Carousel (Embla), Fallback Image
Utility	Form (react-hook-form adapter), Label, Toggle

Theme System

Themes are implemented via Tailwind's dark mode and custom data-* attributes on . The AccessibilityProvider writes data-theme="high_contrast" (for example) on theme change. CSS variables in globals.css define colour tokens per theme, allowing the entire UI to shift without component-level JavaScript.

Animation

Framer Motion handles page transitions, card entrance animations, and modal overlays. When reduced_motion is enabled in accessibility settings, all motion variants resolve to no-op (opacity-only) transitions to respect user preferences and avoid triggering vestibular disorders.

17 · Environment Variables & Configuration

Variable	Required	Purpose
NEXT_PUBLIC_SUPABASE_URL	Yes	Supabase project URL (e.g. https://.supabase.co). Public — safe to expose to browser.
NEXT_PUBLIC_SUPABASE_ANON_KEY	Yes	Supabase anonymous key. Public — RLS enforces access control.
SUPABASE_SERVICE_ROLE_KEY	Yes	Service role key. SERVER-SIDE ONLY. Bypasses RLS. Never expose to browser.
SMTP_HOST	Yes	SMTP server hostname (smtp.gmail.com for Gmail).
SMTP_PORT	Yes	SMTP port. Use 587 for TLS (STARTTLS).
SMTP_USER	Yes	Gmail address used as SMTP sender.
SMTP_PASS	Yes	16-character Gmail App Password (not the Gmail login password).
EMAIL_FROM	Yes	Sender display name and address: "ACESS"

Never commit .env.local to version control. The SUPABASE_SERVICE_ROLE_KEY grants full database access bypassing all RLS policies. Treat it as a root database password. In Vercel, set it as an encrypted environment variable scoped to Production only.

18 · Deployment & Setup Guide

18.1 Prerequisites

- Node.js 18 or higher
- npm (bundled with Node.js)
- A Supabase project (free tier is sufficient)
- A Gmail account with 2-Step Verification enabled (for SMTP)

18.2 Local Development Setup

```
# 1. Clone the repository
git clone https://github.com/aliff1024/ACESS
cd ACESS

# 2. Install dependencies
npm install

# 3. Configure environment variables
# Create .env.local and populate all required variables (see Section 17)

# 4. Run database migrations (in Supabase SQL Editor, in order):
# scripts/scheme_script_corrected.sql
# scripts/create-notifications.sql
# scripts/create-favorites.sql
# scripts/fix-admin-rls.sql
# scripts/system_content_migration.sql
# supabase/migrations/20260510_password_reset_tokens.sql
# supabase/migrations/20260510_add_chapters_templates_checkpoints.sql
# supabase/migrations/20260510_add_system_courses.sql

# 5. Start the development server
npm run dev # http://localhost:3000
```

18.3 Gmail App Password Setup

- Step 1: Sign in to your Google Account.
- Step 2: Navigate to Security → 2-Step Verification and enable it if not already active.
- Step 3: Go to Security → App passwords.
- Step 4: Select "Other" from the app dropdown and enter "ACESS" as the name.
- Step 5: Click Generate and copy the 16-character password shown.

- Step 6: Set SMTP_PASS to this value in .env.local (and in Vercel for production).
- Step 7: Set SMTP_USER to your Gmail address.

18.4 Vercel Deployment

```
# Option A: Vercel CLI (recommended)
npm install -g vercel
vercel login
vercel link --project "acess"

# Set each environment variable (repeat for all 8 variables):
vercel env add NEXT_PUBLIC_SUPABASE_URL production
vercel env add NEXT_PUBLIC_SUPABASE_ANON_KEY production
vercel env add SUPABASE_SERVICE_ROLE_KEY production
vercel env add SMTP_HOST production
vercel env add SMTP_PORT production
vercel env add SMTP_USER production
vercel env add SMTP_PASS production
vercel env add EMAIL_FROM production

# Deploy to production
vercel deploy --prod --yes

# Option B: GitHub Integration
# 1. Push code to GitHub main branch
# 2. Import repo in Vercel dashboard → auto-deploys on every push
# 3. Set env vars in: Project Settings → Environment Variables
```

18.5 Key Resources

Resource	URL
Live Application	https://acess-tau.vercel.app
GitHub Repository	https://github.com/aliff1024/ACESS
Supabase Dashboard	https://supabase.com/dashboard/project/kdlryupwmydirgvxuixd
Vercel Dashboard	https://vercel.com/aliff1024s-projects/acess

19 · Troubleshooting

Build fails: "Can't resolve 'nodemailer'"

Run npm install to ensure all dependencies are installed. This typically occurs after a fresh clone without running the install step.

SMTP not sending emails

Verify SMTP_PASS is the 16-character App Password and not the Gmail account login password. Confirm 2-Step Verification is enabled. Test the connection using the verifySmtpConnection() helper in src/lib/email.ts by calling it from an API route temporarily.

Auth middleware blocking public routes

Add the new path to the PUBLIC_ROUTES array in src/proxy.ts. Ensure the matcher regex at the root proxy.ts does not exclude your new route.

forgot-password / reset-password not working

Confirm the password_reset_tokens table exists (run the migration). Verify SUPABASE_SERVICE_ROLE_KEY is set and correct. Check that both /forgot-password and /reset-password (plus their API routes) are in PUBLIC_ROUTES.

RLS errors in Supabase logs

RLS denials appear as 'permission denied for table X'. Ensure the correct policy exists for the operation (SELECT/INSERT/UPDATE/DELETE) and actor (authenticated user vs service role). Check that the query is using the correct client (browser client for user ops, service role client for admin ops).

Supabase realtime notifications not firing

Check that the relevant PostgreSQL trigger exists in the database. Verify the Supabase realtime extension is enabled on the notifications table in the Supabase dashboard (Database → Replication).

Accessibility settings not persisting

Confirm the user_accessibility_settings table exists and the user has a row (created on first save). Check that the updateSetting() call in AccessibilityProvider is not being debounced away before the page navigation.

Course thumbnails showing broken image

The Fallback Image component handles this gracefully by substituting a placeholder. Verify the course-assets bucket policy allows the uploading educator to write files. Check the URL stored in courses.thumbnail_url is a valid Supabase Storage public URL.

20 · Dependency Reference

Production Dependencies

Package	Version (approx.)	Purpose
next	16.2.3	Full-stack React framework with App Router, SSR, and API routes
react / react-dom	19.x	UI rendering library
typescript	5.x	Static typing across the entire codebase
@supabase/supabase-js	latest	Database queries, auth, storage, and realtime
@supabase/ssr	latest	Cookie-based server-side session management for Next.js
tailwindcss	4.x	Utility-first CSS framework
@radix-ui/*	various	Headless, accessible UI primitives (shadcn/ui base)
class-variance-authority	latest	Type-safe component variant definitions (cva)
clsx + tailwind-merge	latest	Conditional class merging utility (cn())
framer-motion	latest	Declarative animation library for page transitions and micro-interactions
recharts	latest	Composable chart library for analytics dashboards
react-hook-form	latest	Performant, flexible forms with minimal re-renders
zod	latest	TypeScript-first schema validation for form inputs
nodemailer	latest	SMTP email sending for password reset flow
sonner	latest	Toast notification library
lucide-react	latest	Consistent, tree-shakeable icon set
embla-carousel-react	latest	Touch-friendly carousel for lesson assets
next-themes	latest	Theme detection and switching (used with Tailwind dark mode)

Node.js Built-ins Used

Module	Used For
crypto	Secure random token generation for password reset (crypto.randomBytes(32).toString('hex'))

This document was generated from the ACESS platform source documentation. For the latest updates, refer to the GitHub repository at https://github.com/aliff1024/ACESS. Live deployment: https://acess-tau.vercel.app.